UNIVERSIDAD TECNOLÓGICA DEL VALLE DE TOLUCA

Dirección de Carrera de Tecnologías de la Información y Comunicación

Ingeniería en Tecnologías de la Información

Área Desarrollo y Gestión de Software

DESARROLLO MÓVIL INTEGRAL

Actividad 22

Lara Rodríguez Joel Alejandro

Grupo

IDGS – 101

Cuatrimestre

10º

Lugar

Santa María Atarasquillo, Lerma, México.

Fecha de Entrega

Octubre del 2026

1. Descripción de la solución

Mantenimiento U es una aplicación móvil desarrollada con React Native y Expo SDK 58 para Android e iOS. Permite iniciar sesión mediante un servicio remoto, registrar problemas de infraestructura, adjuntar fotografías, consultar reportes propios, revisar su seguimiento y trabajar temporalmente sin conexión. La solución se conecta con una API REST desplegada en la nube y aplica permisos por usuario y rol.

Funcionalidades implementadas

- Autenticación remota con correo y contraseña; token almacenado en Expo SecureStore.
- Alta de reportes con categoría, prioridad, ubicación, título, descripción, fecha y evidencia fotográfica desde cámara o galería.
- Consulta y búsqueda de los reportes autorizados para el usuario.
- Seguimiento por estados: Recibido, En revisión, En proceso y Resuelto.
- Panel móvil con indicadores e historial.
- Permisos de estudiante y administrador aplicados por la API.
- Cola local para reportes creados sin conexión y sincronización posterior.
- Proyecto Expo SDK 58 preparado para ejecutarse en Expo Go y generar APK o AAB con EAS Build.

2. Respuestas de la evaluación teórica

¿Cómo se integran una aplicación móvil y la nube?

La aplicación envía solicitudes HTTPS a endpoints de una API REST. El servidor valida el token, los permisos y los datos; después consulta o modifica la información y devuelve una respuesta JSON con un código HTTP. La interfaz procesa esa respuesta y muestra el resultado al usuario.

¿Qué función cumplen JSON y los métodos HTTP?

JSON representa los datos intercambiados. GET consulta información, POST crea recursos, PUT o PATCH actualiza datos y DELETE elimina recursos cuando está permitido. Los códigos 200, 201, 400, 401, 403, 404 y 500 indican el resultado de la operación.

¿Cómo se protege la integración?

Se utiliza HTTPS, autenticación con token de duración limitada, contraseñas derivadas con PBKDF2, validación en servidor, autorización por propietario y rol, límites de tamaño, encabezados de seguridad y manejo controlado de errores.

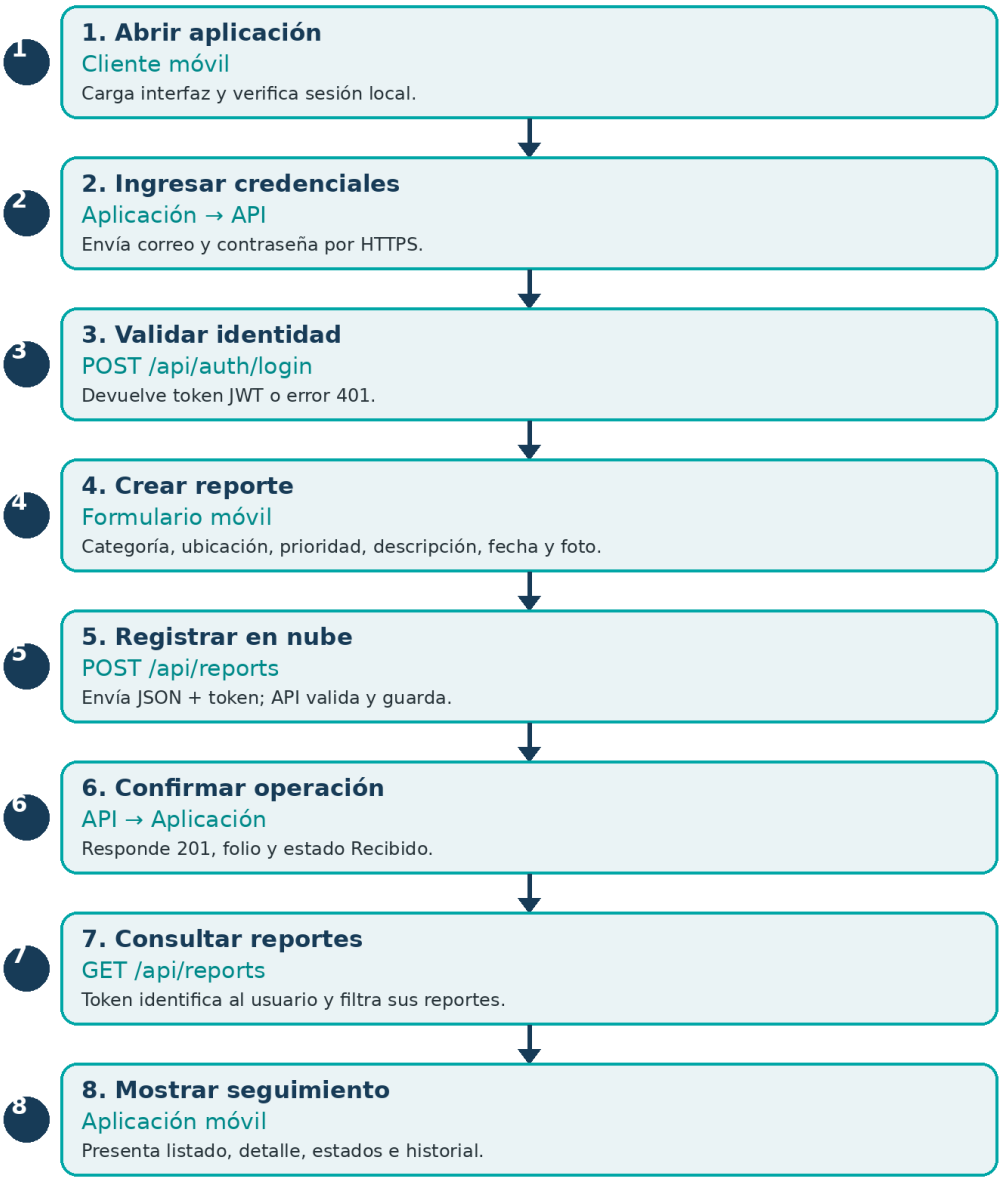
3. Identificación y justificación de servicios tecnológicos

Componente	Tecnología	Justificación
Aplicación móvil	React Native + Expo SDK 58	Genera una aplicación para Android/iOS y permite probarla en Expo Go.
API REST	Node.js	Recibe solicitudes, valida datos, aplica reglas de negocio y devuelve JSON.
Servicio en la nube	Render Web Service	Publica la aplicación y la API mediante HTTPS.
Datos	Archivo JSON en prototipo	Permite demostrar persistencia; para producción se recomienda PostgreSQL administrado.

Autenticación	JWT firmado + PBKDF2	Mantiene sesiones limitadas sin enviar la contraseña en cada consulta.
Evidencia	Imagen asociada al reporte	El prototipo la transmite a la API; en producción debe guardarse en almacenamiento de objetos.
Trabajo sin conexión	Service Worker + almacenamiento local	Conserva la interfaz y encola reportes hasta recuperar conectividad.
Versionamiento	Git y GitHub	Registra cambios y conserva la Actividad 22 en un repositorio independiente.

La arquitectura seleccionada es cliente-servidor en capas: presentación móvil, API y persistencia. Esta separación facilita el mantenimiento, las pruebas, el cambio de base de datos y la incorporación futura de clientes Android o iOS nativos.

4. Diseño del proceso de integración móvil–nube



En cada solicitud protegida, la aplicación envía Authorization: Bearer <token>. El servidor identifica al usuario, verifica su rol y limita los datos al propietario. Si la validación falla devuelve un error controlado; si es correcta devuelve JSON con el recurso solicitado.

5. Riesgos y medidas de codificación segura

Riesgo	Impacto	Medida preventiva
Credenciales expuestas	Robo de cuentas o control del servidor.	No incluir secretos administrativos en el cliente; usar variables de entorno y contraseñas derivadas con PBKDF2.
Comunicación interceptada	Lectura o modificación de datos.	Usar exclusivamente HTTPS/TLS y rechazar conexiones inseguras.
Fuerza bruta	Intentos repetidos de acceso.	Limitar solicitudes de inicio de sesión y responder con mensajes genéricos.
Acceso a reportes ajenos	Pérdida de confidencialidad.	Verificar propietario y rol en cada endpoint; devolver 403 cuando no exista autorización.
Entrada maliciosa	Inyección, XSS o datos corruptos.	Validar tipo, longitud, valores permitidos y contenido en el servidor.
Archivos peligrosos o grandes	Consumo de recursos o contenido ejecutable.	Aceptar solo imágenes permitidas, limitar a 2 MB y aplicar una política CSP.
Robo o reutilización del token	Suplantación de sesión.	Firmar el token, limitar su vigencia, cerrar sesión y usar almacenamiento seguro en una versión nativa.
Servicio sin conexión	Pérdida del reporte o mala experiencia.	Encolar el reporte localmente, informar el estado y reintentar al recuperar conexión.

Medida adicional

Se conserva un historial de cambios de estado con fecha y usuario responsable. Esta trazabilidad es necesaria para detectar modificaciones indebidas, comprobar quién atendió cada reporte y facilitar una auditoría.

6. Operaciones de integración

Operación	Método	Endpoint	Datos	Respuesta
Iniciar sesión	POST	/api/auth/login	Correo y contraseña	200 + token / 401
Consultar perfil	GET	/api/auth/me	Token	200 + usuario
Panel	GET	/api/dashboard	Token	200 + indicadores
Registrar reporte	POST	/api/reports	Token + JSON del reporte	201 + folio
Consultar reportes	GET	/api/reports	Token + filtros	200 + lista autorizada
Consultar detalle	GET	/api/reports/:id	Token	200 / 403 / 404
Cambiar estado	PATCH	/api/reports/:id/status	Token admin + estado	200 / 403
Agregar comentario	POST	/api/reports/:id/comments	Token admin + texto	201 / 403

Preguntas de análisis

¿Por qué no se deben incluir credenciales administrativas en la aplicación móvil? Porque el paquete puede inspeccionarse. Un atacante podría extraerlas y obtener privilegios sobre todos los datos. Los secretos deben permanecer en el servidor y la aplicación debe usar permisos mínimos.

¿Qué validaciones deben realizarse antes de enviar un reporte? Sesión vigente; categoría, prioridad y edificio permitidos; título, ubicación y descripción obligatorios; longitudes máximas; fecha válida; imagen con tipo permitido y tamaño menor a 2 MB.

¿Qué debe suceder si el servicio en la nube no está disponible? La aplicación debe informar la falta de conexión, conservar el reporte en una cola local, evitar duplicados y reintentar la sincronización cuando vuelva la conectividad.

7. Propuesta de pruebas y resultados esperados

Prueba	Procedimiento	Resultado esperado	Resultado
Autenticación correcta	Ingresar credenciales válidas.	Token y panel del usuario.	Aprobada
Autenticación incorrecta	Ingresar una contraseña inválida.	Respuesta 401 sin revelar qué dato falló.	Aprobada
Registro de reporte	Completar campos y enviar.	Respuesta 201, folio y estado Recibido.	Aprobada
Consulta de información	Solicitar la lista con token válido.	Solo aparecen reportes autorizados.	Aprobada
Acceso no autorizado	Un estudiante solicita un reporte ajeno.	Respuesta 403 y ningún dato del reporte.	Aprobada
Sin conexión	Desactivar la red y crear un reporte.	Se guarda en cola y se sincroniza al volver la red.	Implementada
Validación de evidencia	Intentar adjuntar un archivo mayor a 2 MB.	Se rechaza y se muestra un mensaje claro.	Implementada

Las cinco pruebas automatizadas de la API finalizaron correctamente. El proyecto móvil superó las 20 verificaciones de Expo Doctor y Metro generó correctamente los paquetes para Android y web. También se verificaron autenticación, panel, alta de un reporte, consulta y manejo de credenciales incorrectas.

8. Evidencia de la aplicación funcionando



Figura 1. Pantalla móvil de autenticación remota.



Figura 2. Autenticación exitosa y panel móvil con indicadores.



Figura 3. Formulario móvil para registrar un problema.

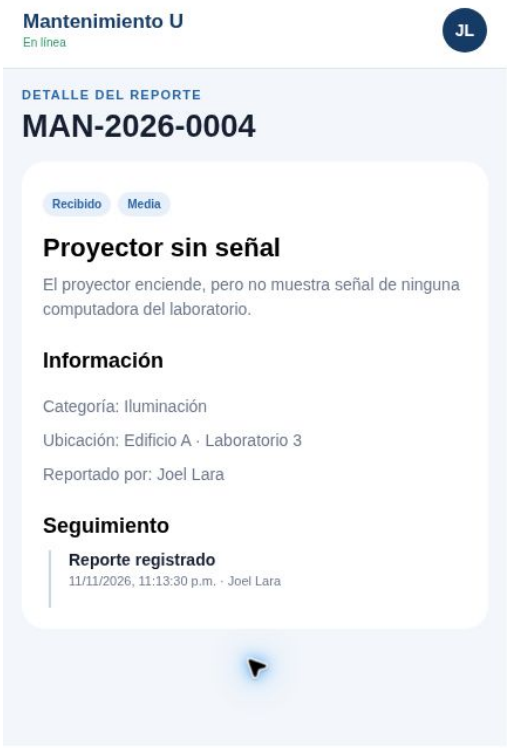


Figura 4. Reporte enviado al servicio y folio generado.



Figura 5. Consulta móvil de reportes del usuario.



Figura 6. Manejo de credenciales incorrectas.

9. Conclusión personal

Considero que la codificación segura debe formar parte del diseño desde el inicio, porque una aplicación móvil conectada a la nube maneja identidades, datos y operaciones que pueden ser interceptados o modificados. En este caso no basta con ocultar opciones en la interfaz: la API debe validar el token, el propietario y el rol en cada solicitud. También es necesario validar los datos en el servidor, limitar los archivos, proteger las contraseñas y utilizar HTTPS. La cola sin conexión mejora la disponibilidad, pero debe sincronizarse con cuidado para evitar duplicados. La solución desarrollada demuestra el ciclo completo de autenticación, registro, consulta y seguimiento, y deja una base escalable para integrar una base de datos y almacenamiento de objetos en un entorno de producción.

10. Fuentes y enlaces del proyecto

- **OWASP Mobile Application Security:** https://mas.owasp.org/
- **Expo SDK 58:** https://docs.expo.dev/versions/v58.0.0/
- **MDN: métodos HTTP:** https://developer.mozilla.org/es/docs/Web/HTTP/Methods
- **JWT:** https://www.rfc-editor.org/rfc/rfc7519
- **Vista móvil desplegada:** https://mantenimiento-u-actividad22.onrender.com/mobile/index.html
- **Repositorio independiente:** https://github.com/JoelLara10/actividad-22-mantenimiento-universitario